

Global Common Subexpression Elimination

John Cocke

IBM CORPORATION

INTRODUCTION

When considering compiler optimization, there are two questions that immediately come to mind; one, why and to what extent is optimization necessary and two, to what extent is it possible.

When considering the second question, one might immediately become discouraged since it is well known that the program equivalency problem is recursively unsolvable. It is, of course, clear from this that there will never be techniques for generating a completely optimum program. These unsolvability results, however, do not preclude the possibility of ad hoc techniques for program improvement or even a partial theory which produces a class of equivalent programs optimized in varying degrees.

The reasons why optimization is required seem to me to fall in two major categories. The first I will call "local" and the second "global".

Local optimizations are required due to particular peculiarities or characteristics of a computer and, in general, probably do not lend themselves well to any theory of optimization, but are desirable since significant machine performance can be gained by attention to details of this sort. There are many interesting tricks that I could discuss, however, due to the lack of any generality, I will not.

Global optimization is required due to the fact that the known techniques for code generation essentially utilize only local information when selecting code, and while it may be possible to make this global information available at the time of code generation, the complexity of such a procedure seems so great that a better strategy seems to be to separate these global considerations from the code generation and give it the name optimization. Elimination of redundant computations, index allocation and reduction in strength are examples of optimizations that require global information to achieve good object code. Moreover, there are general techniques for gathering the necessary information to do these optimizations.

In the talk which follows, I will give an example of how to eliminate common subexpressions, developing with this general techniques for analyzing the flow of programs and determining the conditions

sufficient for computation elimination. These techniques are also applicable (with modifications) to constant propagation, reduction in strength and register allocation. It is felt that these procedures form the basis of a beginning of a formal theory for a class of optimization techniques.

The object of the procedure to be described is to eliminate computations redundant in one of two senses. In the first place, we aim to delete those computations which have been previously performed, secondly we aim when possible to move computations from their original positions to less frequently executed regions. There exist numerous well known methods for the elimination of redundant computations within basic blocks (i.e. sections of a program which have the property that if any instruction in the block is executed all must be). We shall not discuss these methods but will consider the more general case of computation elimination in the presence of flow.

Accordingly, we assume the program PR to be optimized to be written in a source language which is such that the basic blocks of PR and the flow of control in PR can be determined. This allows us to represent PR by a directed graph whose nodes are the basic blocks comprising PR and containing its computations. We will assume that the basic blocks $i = 1, 2, \dots, n$ are given, and associated with each basic block i there are two sets: $S(i)$ the set of immediate successors of the i -th block and $P(i)$ the set of immediate predecessors. The j -th computation $F_j(X_1, \dots, X_m)$ within a basic block is to be represented in some standard form "intermediate language" let us say by listing the operation to be performed together with their arguments. For example, if $A*B+C*D$ appeared in PR the three following computations would appear in the intermediate language.

*A,B: α	The arguments of a commutative operation will
*C,D: β	occur in some standard lexical order in the inter-
+ α, β :	mediate language form of each program PR, so that $A+B$
	and $B+A$ will be represented by some expression $+A, B$.

However, the method of parse will determine the computations which are listed in the intermediate language form of a translated program, and no attempt will be made to use the associative law to optimize those computations to be eliminated. For example, $A+B+C$ will generate one but not both of the two possible code sequences

+A,B: α	Two computations which are represented by the
+ α, C : β	same intermediate language sequence are called
or	formally identical. The principal object of our
+B,C: α	optimization procedures is to determine which of a
+A, α : β	set of formally identical computations is in fact
	redundant.

The following examples show the basic problem to be faced. Consider the sequences

A := B*C	In the first case the second calculation of B*C
D := B*C	is redundant. In the second sequence, due to the
and	assignment to B, the second computation of B*C is
	not redundant.

A := B*C	As previously mentioned our procedures are
B := Z	concerned with the elimination of redundant compu-
D := B*C	tations in the presence of control flow.
	Our basic method determines, for each

computation C occurring in a given basic block B whether or not C would be redundant if a formally identical computation occurred at the beginning of the block. With this information made available, there exist many simple procedures (not to be discussed) for performing the actual elimination.

To make the above determination we introduce Boolean variables X_j^i having the following meaning

$X_j^i = 1$ if the j computation is redundant on exit from the i-th block;

$X_j^i = 0$ otherwise,

and put

$Y_j^i = \bigwedge_{K \in P(i)} X_j^K$, where $P(i)$ is the set of predecessor blocks of the i-th block.

Then $Y_j^i = 1$ if and only if the j-th computation is redundant at the entrance to the i-th block. (We denote the boolean and function by $\times$ and the boolean or by $+$.)

The following boolean constants can be determined from the code constituting a basic block:

$a_j^i = 1$ means that there is no assignment of any argument of the j-th computation within the i-th block; $a_j^i = 0$ otherwise.

$b_j^i = 1$ means that there is in the i-th block a computation of the j-th expression which follows all assignments of any of its arguments; $b_j^i = 0$ otherwise.

The fundamental quantities X_j^i may be obtained as the solution of the following system of boolean equations:

(*) $X_j^i = a_j^i \bigwedge_{K \in P(i)} X_j^K + b_j^i$. At first sight solution of these equations for each formal quantity occurring in a program would seem to

involve a prodigious amount of work. However, the method to be described below solves these equations very economically. Its advantages derive from two facts. First, due to the sparseness of the boolean matrices typically occurring in the equations (*), a flow analysis can be made which arranges the necessary Gaussian elimination in such a way that the work is minimized (as a secondary benefit of our elimination method, one determines those expressions which can be moved from higher to lower frequency regions). Secondly the solution procedure uses only the and and or functions, and thus allows computation to proceed in parallel for many formal quantities at once, since most computers perform these operations in parallel over the bits of an entire word.

FLOW ANALYSIS

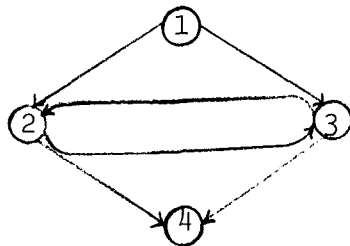
The special Gaussian elimination procedure necessary to solve the boolean equations (*) for a program PR is based on an analysis of the flow graph; the method uses an entity called an Interval. Intervals are found as follows. (A more complete and elementary description of this process can be found in John Cocke and J. T. Schwartz (1970), Programming Languages and Their Compilers: Preliminary Notes, Courant Institute of Mathematical Sciences, New York University.)

Given a set of nodes S of a directed graph let $F(S) = S \cup \{p \mid P(p) \subseteq S\}$. We define the interval of a point p as $I_p = F^\infty(\{p\})$. The point p is called the head of I_p . We note two properties of I_p . First, p back-dominates I_p ; second, $I_p - \{p\}$ is cycle-free.

Our flow-analysis proceeds as follows:

1. Find all maximal intervals (i.e. those intervals which are not a proper subset of another interval).
2. Form a derived graph according to the following rule: the intervals of the first graph are the nodes of the derived graph; the connectivity of the derived graph is that induced from the first graph.
3. Repeat this process until it is no longer possible to achieve a reduced number of nodes.

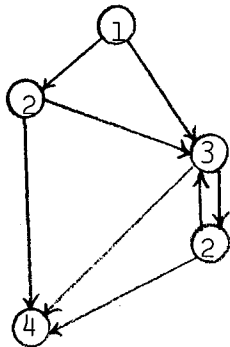
Studies have shown that approximately 90% of all Fortran programs will be reduced by this iterative derivation procedure to a single node. However there exist graphs which will not reduce to a single node. Here is a simple example:



This graph has the property that the cycle consisting of the nodes 2 and 3 is entered in more than a single node.

This difficulty can be eliminated by a method known as node splitting (for complete description, see John Cocke and Raymond E. Miller (1969), "Some Analysis Techniques for Optimizing Computer Programs," Proceedings, System Sciences, Hawaii, January, 1969).

The "split" version of the graph shown above appears as follows:



By duplicating node 2 of the previous graph we have obtained a graph which will reduce to a single point and which represents a program equivalent to that represented by the previous graph. Using this splitting process iteratively, we may reduce all graphs to a single node.

EFFICIENT GAUSSIAN ELIMINATION

For simplicity, we shall in this final section consider only programs whose graphs reduce to single points by repeated application of the above-described derivation procedure, without any node-splitting. In this case, the fundamental boolean equations (*) given above may be solved as follows.

Pick any interval I of the program graph; set up the a and b bit vectors as determined by the basic blocks. Allow one bit position for each formal quantity of the program, using as many words as required for this purpose. Make a vector Z of all zeros and a vector $\emptyset$ of all ones. Trace through the interval twice starting with its head.

Proceed to a node j with the X_j^i vector has been calculated for all its predecessors; calculate X_j^i by the formula $X_j^i = a_j^i \times \prod_{i \in P(j)} X_j^i + b_j^i$.

On the first pass, ignore the cycles which enter the head of the interval but on the second use the values calculated on the first. (Due to the monotonicity of the equations (*) two passes are sufficient for the solution.) Now note we have two values for X_j^i , one obtained by starting with Z , the other obtained by starting with $\emptyset$. Call them ${}^0X_j^i$ and ${}^1X_j^i$. Note now that I can be considered as a basic block for which ${}^1X_j^i$ corresponds to the a_j^i vector and the ${}^0X_j^i$ corresponds to the b_j^i at the exits of the interval. Moreover the ${}^0X_j^i$, ${}^1X_j^i$ values attached by our process to the j -th formal expression have the following interpretation: If

${}^1X_j^i = 1$	${}^0X_j^i = 1$	the expression can be eliminated; if
${}^1X_j^i = 0$	${}^0X_j^i = 0$	the expression cannot be eliminated;
${}^1X_j^i = 0$	${}^0X_j^i = 1$	is impossible; and
${}^1X_j^i = 1$	${}^0X_j^i = 0$	means that the expression can either be moved

out of the region or that it can be eliminated by computations outside this interval. In this case if it is an essential node (that is, a node N which must be entered each time control enters), then the expression can be moved out of the interval, an appropriate optimization because the frequency of execution of N is at least as high as that of a location just before the entrance to the interval. Moreover in this case we need not fear causing extraneous interrupt conditions. In case the node is not essential, then the elimination is deferred until the value of X_j^i is determined for the entrance to the interval.

Our overall procedure should now be clear. Start to process the intervals found in the first pass of the interval analysis obtaining a and b vectors for these intervals by the process described. Proceed now to process successive derived in the order in which they were formed by the interval analysis. If the program is not a subroutine only a zero vector is required for the initial input. In case of a subroutine, the deferred computations can be saved for possible elimination when the calling program is processed.

It might be noted that interval analysis can also be used for matrix inversion of sparse matrices which correspond to graphs which can be reduced by the interval process without excessive node splitting.